

Binary Database: Transaction and Vertical Format

D	A	B	C	D	E
1	1	1	0	1	1
2	0	1	1	0	1
3	1	1	0	1	1
4	1	1	1	0	1
5	1	1	1	1	1
6	0	1	1	1	0

Binary Database

t	$i(t)$
1	$ABDE$
2	BCE
3	$ABDE$
4	$ABCE$
5	$ABCDE$
6	BCD

Transaction Database

$t(x)$				
A	B	C	D	E
1	1	2	1	1
3	2	4	3	2
4	3	5	5	3
5	4	6	6	4
	5			5
	6			

Vertical Database

This dataset D has 5 items, $\mathcal{I} = \{A, B, C, D, E\}$ and 6 tids $\mathcal{T} = \{1, 2, 3, 4, 5, 6\}$.

The the first transaction is $\langle 1, \{A, B, D, E\} \rangle$, where we omit item C since $(1, C) \notin D$. Henceforth, for convenience, we drop the set notation for itemsets and tidsets. Thus, we write $\langle 1, \{A, B, D, E\} \rangle$ as $\langle 1, ABDE \rangle$.

Frequent Itemset Mining

In many applications one is interested in how often two or more objects of interest co-occur, the so-called *itemsets*.

The prototypical application was *market basket analysis*, that is, to mine the sets of items that are frequently bought together at a supermarket by analyzing the customer shopping carts (the so-called “market baskets”).

Frequent itemset mining is a basic exploratory mining task, since the basic operation is to find the co-occurrence, which gives an estimate for the joint probability mass function.

Once we mine the frequent sets, they allow us to extract *association rules* among the itemsets, where we make some statement about how likely are two sets of items to co-occur or to conditionally occur.

Frequent Itemsets: Terminology

Itemsets: Let $\mathcal{I} = \{x_1, x_2, \dots, x_m\}$ be a set of elements called *items*. A set $X \subseteq \mathcal{I}$ is called an *itemset*. An itemset of cardinality (or size) k is called a k -itemset. Further, we denote by $\mathcal{I}^{(k)}$ the set of all k -itemsets, that is, subsets of $\mathcal{I}$ with size k .

Tidsets: Let $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ be another set of elements called transaction identifiers or *tids*. A set $T \subseteq \mathcal{T}$ is called a *tidset*. Itemsets and tidsets are kept sorted in lexicographic order.

Transactions: A *transaction* is a tuple of the form $\langle t, X \rangle$, where $t \in \mathcal{T}$ is a unique transaction identifier, and X is an itemset.

Database: A binary database $\mathbf{D}$ is a binary relation on the set of tids and items, that is, $\mathbf{D} \subseteq \mathcal{T} \times \mathcal{I}$. We say that tid $t \in \mathcal{T}$ *contains* item $x \in \mathcal{I}$ iff $(t, x) \in \mathbf{D}$. In other words, $(t, x) \in \mathbf{D}$ iff $x \in X$ in the tuple $\langle t, X \rangle$. We say that tid t *contains* itemset $X = \{x_1, x_2, \dots, x_k\}$ iff $(t, x_i) \in \mathbf{D}$ for all $i = 1, 2, \dots, k$.

Database Representation

Let 2^X denote the powerset of X , that is, the set of all subsets of X . Let $\mathbf{i} : 2^{\mathcal{T}} \rightarrow 2^{\mathcal{I}}$ be a function, defined as follows:

$$\mathbf{i}(T) = \{x \mid \forall t \in T, t \text{ contains } x\}$$

where $T \subseteq \mathcal{T}$, and $\mathbf{i}(T)$ is the set of items that are common to *all* the transactions in the tidset T . In particular, $\mathbf{i}(t)$ is the set of items contained in tid $t \in \mathcal{T}$.

Let $\mathbf{t} : 2^{\mathcal{I}} \rightarrow 2^{\mathcal{T}}$ be a function, defined as follows:

$$\mathbf{t}(X) = \{t \mid t \in \mathcal{T} \text{ and } t \text{ contains } X\} \quad (1)$$

where $X \subseteq \mathcal{I}$, and $\mathbf{t}(X)$ is the set of tids that contain *all* the items in the itemset X . In particular, $\mathbf{t}(x)$ is the set of tids that contain the single item $x \in \mathcal{I}$.

The binary database $\mathbf{D}$ can be represented as a *horizontal* or *transaction database* consisting of tuples of the form $\langle t, \mathbf{i}(t) \rangle$, with $t \in \mathcal{T}$.

The binary database $\mathbf{D}$ can also be represented as a *vertical* or *tidset database* containing a collection of tuples of the form $\langle x, \mathbf{t}(x) \rangle$, with $x \in \mathcal{I}$.

Support and Frequent Itemsets

The *support* of an itemset X in a dataset $\mathbf{D}$, denoted $sup(X)$, is the number of transactions in $\mathbf{D}$ that contain X :

$$sup(X) = |\{t \mid \langle t, \mathbf{i}(t) \rangle \in \mathbf{D} \text{ and } X \subseteq \mathbf{i}(t)\}| = |\mathbf{t}(X)|$$

The *relative support* of X is the fraction of transactions that contain X :

$$rsup(X) = \frac{sup(X)}{|\mathbf{D}|}$$

It is an estimate of the *joint probability* of the items comprising X .

An itemset X is said to be *frequent* in $\mathbf{D}$ if $sup(X) \geq minsup$, where $minsup$ is a user defined *minimum support threshold*.

The set $\mathcal{F}$ denotes the set of all frequent itemsets, and $\mathcal{F}^{(k)}$ denotes the set of frequent k -itemsets.

Frequent Itemsets

Minimum support: $minsup = 3$

t	$i(t)$
1	ABDE
2	BCE
3	ABDE
4	ABCE
5	ABCDE
6	BCD

Transaction Database

sup	itemsets
6	B
5	E, BE
4	$A, C, D, AB, AE, BC, BD, ABE$
3	$AD, CE, DE, ABD, ADE, BCE, BDE, ABDE$

Frequent Itemsets

The 19 frequent itemsets shown in the table comprise the set $\mathcal{F}$. The sets of all frequent k -itemsets are

$$\mathcal{F}^{(1)} = \{A, B, C, D, E\}$$

$$\mathcal{F}^{(2)} = \{AB, AD, AE, BC, BD, BE, CE, DE\}$$

$$\mathcal{F}^{(3)} = \{ABD, ABE, ADE, BCE, BDE\}$$

$$\mathcal{F}^{(4)} = \{ABDE\}$$

Itemset Mining Algorithms: Brute Force

The brute-force algorithm enumerates all the possible itemsets $X \subseteq \mathcal{I}$, and for each such subset determines its support in the input dataset $\mathbf{D}$. The method comprises two main steps: (1) candidate generation and (2) support computation.

Candidate Generation: This step generates all the subsets of $\mathcal{I}$, which are called *candidates*, as each itemset is potentially a candidate frequent pattern. The candidate itemset search space is clearly exponential because there are $2^{|\mathcal{I}|}$ potentially frequent itemsets.

Support Computation: This step computes the support of each candidate pattern X and determines if it is frequent. For each transaction $\langle t, \mathbf{i}(t) \rangle$ in the database, we determine if X is a subset of $\mathbf{i}(t)$. If so, we increment the support of X .

Computational Complexity: Support computation takes time $O(|\mathcal{I}| \cdot |\mathbf{D}|)$ in the worst case, and because there are $O(2^{|\mathcal{I}|})$ possible candidates, the computational complexity of the brute-force method is $O(|\mathcal{I}| \cdot |\mathbf{D}| \cdot 2^{|\mathcal{I}|})$.

Brute Force Algorithm

BruteForce ($D, \mathcal{I}, \text{minsup}$):

```
1  $\mathcal{F} \leftarrow \emptyset$  // set of frequent itemsets
2
3 foreach  $X \subseteq \mathcal{I}$  do
4    $\text{sup}(X) \leftarrow \text{ComputeSupport}(X, D)$ 
5   if  $\text{sup}(X) \geq \text{minsup}$  then
6      $\mathcal{F} \leftarrow \mathcal{F} \cup \{(X, \text{sup}(X))\}$ 
7 return  $\mathcal{F}$ 
```

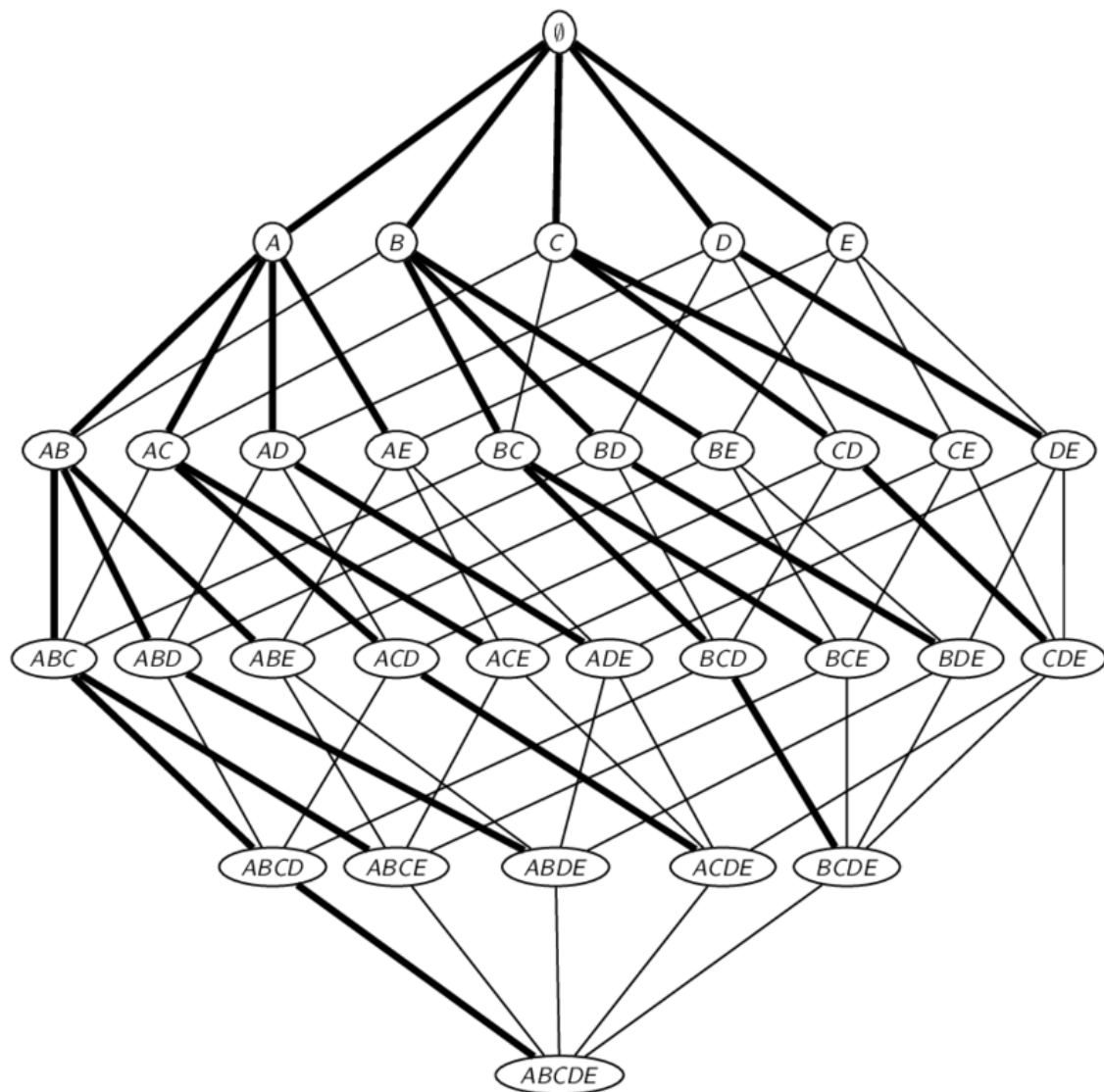
ComputeSupport (X, D):

```
1  $\text{sup}(X) \leftarrow 0$ 
2 foreach  $\langle t, i(t) \rangle \in D$  do
3   if  $X \subseteq i(t)$  then
4      $\text{sup}(X) \leftarrow \text{sup}(X) + 1$ 
5 return  $\text{sup}(X)$ 
```


Itemset lattice and prefix-based search tree

Itemset search space is a lattice where any two itemsets X and Y are connected by a link iff X is an *immediate subset* of Y , that is, $X \subseteq Y$ and $|X| = |Y| - 1$.

Frequent itemsets can be enumerated using either a BFS or DFS search on the *prefix tree*, where two itemsets X, Y are connected by a link iff X is an immediate subset and prefix of Y . This allows one to enumerate itemsets starting with an empty set, and adding one more item at a time.



Level-wise Approach: Apriori Algorithm

If $X \subseteq Y$, then $\text{sup}(X) \geq \text{sup}(Y)$, which leads to the following two observations: (1) if X is frequent, then any subset $Y \subseteq X$ is also frequent, and (2) if X is not frequent, then any superset $Y \supseteq X$ cannot be frequent.

The *Apriori algorithm* utilizes these two properties to significantly improve the brute-force approach. It employs a level-wise or breadth-first exploration of the itemset search space, and prunes all supersets of any infrequent candidate, as no superset of an infrequent itemset can be frequent. It also avoids generating any candidate that has an infrequent subset.

In addition to improving the candidate generation step via itemset pruning, the Apriori method also significantly improves the I/O complexity. Instead of counting the support for a single itemset, it explores the prefix tree in a breadth-first manner, and computes the support of all the valid candidates of size k that comprise level k in the prefix tree.

The Apriori Algorithm

Apriori ($D, \mathcal{I}, \text{minsup}$):

```
1  $\mathcal{F} \leftarrow \emptyset$ 
2  $\mathcal{C}^{(1)} \leftarrow \{\emptyset\}$  // Initial prefix tree with single items
3 foreach  $i \in \mathcal{I}$  do Add  $i$  as child of  $\emptyset$  in  $\mathcal{C}^{(1)}$  with  $\text{sup}(i) \leftarrow 0$ 
4  $k \leftarrow 1$  //  $k$  denotes the level
5 while  $\mathcal{C}^{(k)} \neq \emptyset$  do
6     ComputeSupport ( $\mathcal{C}^{(k)}, D$ )
7     foreach leaf  $X \in \mathcal{C}^{(k)}$  do
8         if  $\text{sup}(X) \geq \text{minsup}$  then  $\mathcal{F} \leftarrow \mathcal{F} \cup \{(X, \text{sup}(X))\}$ 
9         else remove  $X$  from  $\mathcal{C}^{(k)}$ 
10     $\mathcal{C}^{(k+1)} \leftarrow \text{ExtendPrefixTree}(\mathcal{C}^{(k)})$ 
11     $k \leftarrow k + 1$ 
12 return  $\mathcal{F}^{(k)}$ 
```

Apriori Algorithm

ComputeSupport ($\mathcal{C}^{(k)}, D$):

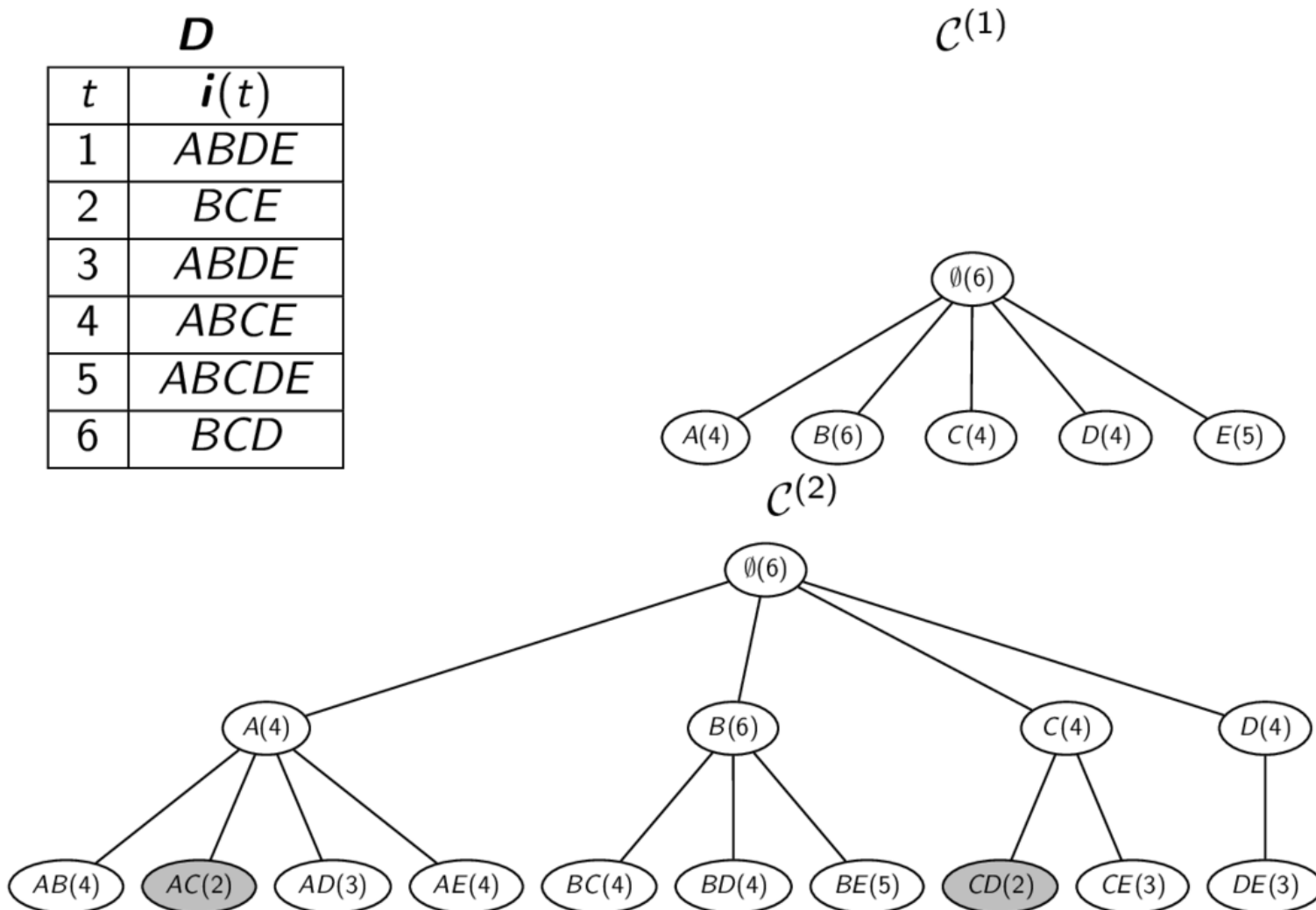
```
1 foreach  $\langle t, i(t) \rangle \in D$  do
2   foreach  $k$ -subset  $X \subseteq i(t)$  do
3     if  $X \in \mathcal{C}^{(k)}$  then  $sup(X) \leftarrow sup(X) + 1$ 
```

ExtendPrefixTree ($\mathcal{C}^{(k)}$):

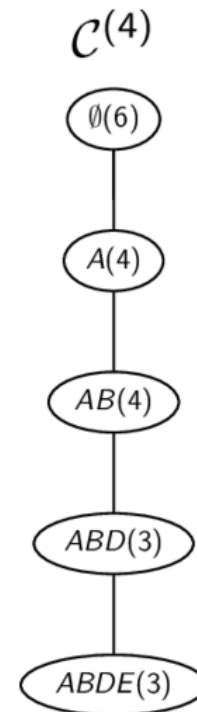
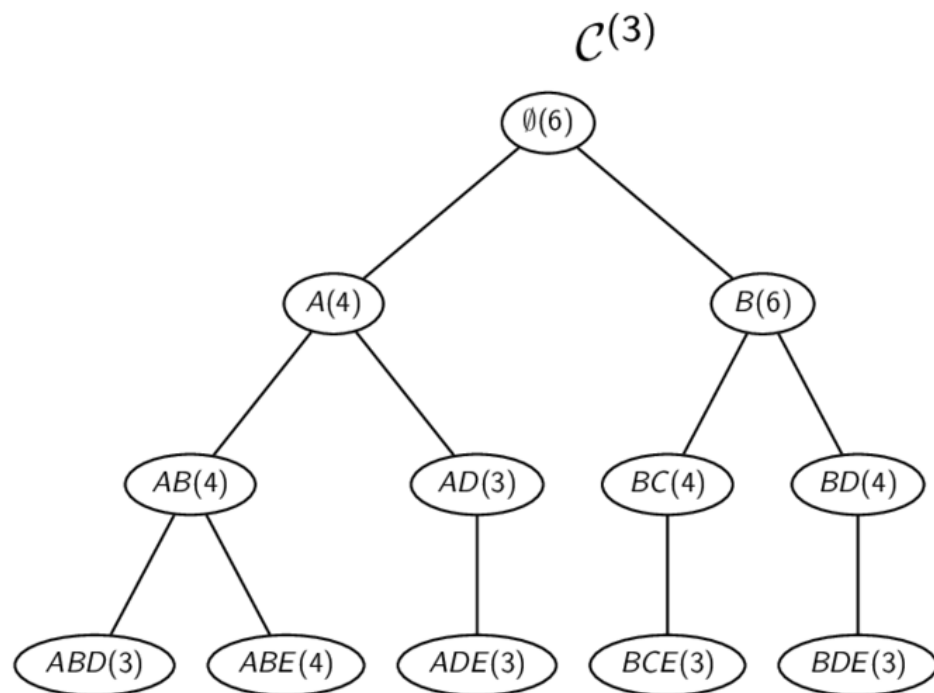
```
1 foreach leaf  $X_a \in \mathcal{C}^{(k)}$  do
2   foreach leaf  $X_b \in \text{sibling}(X_a)$ , such that  $b > a$  do
3      $X_{ab} \leftarrow X_a \cup X_b$ 
4     // prune if there are any infrequent subsets
5     if  $X_j \in \mathcal{C}^{(k)}$ , for all  $X_j \subset X_{ab}$ , such that  $|X_j| = |X_{ab}| - 1$  then
6       Add  $X_{ab}$  as child of  $X_a$  with  $sup(X_{ab}) \leftarrow 0$ 
7   if no extensions from  $X_a$  then
8     remove  $X_a$  and its ancestors with no extensions from  $\mathcal{C}^{(k)}$ 
9 return  $\mathcal{C}^{(k)}$ 
```

Itemset Mining: Apriori Algorithm

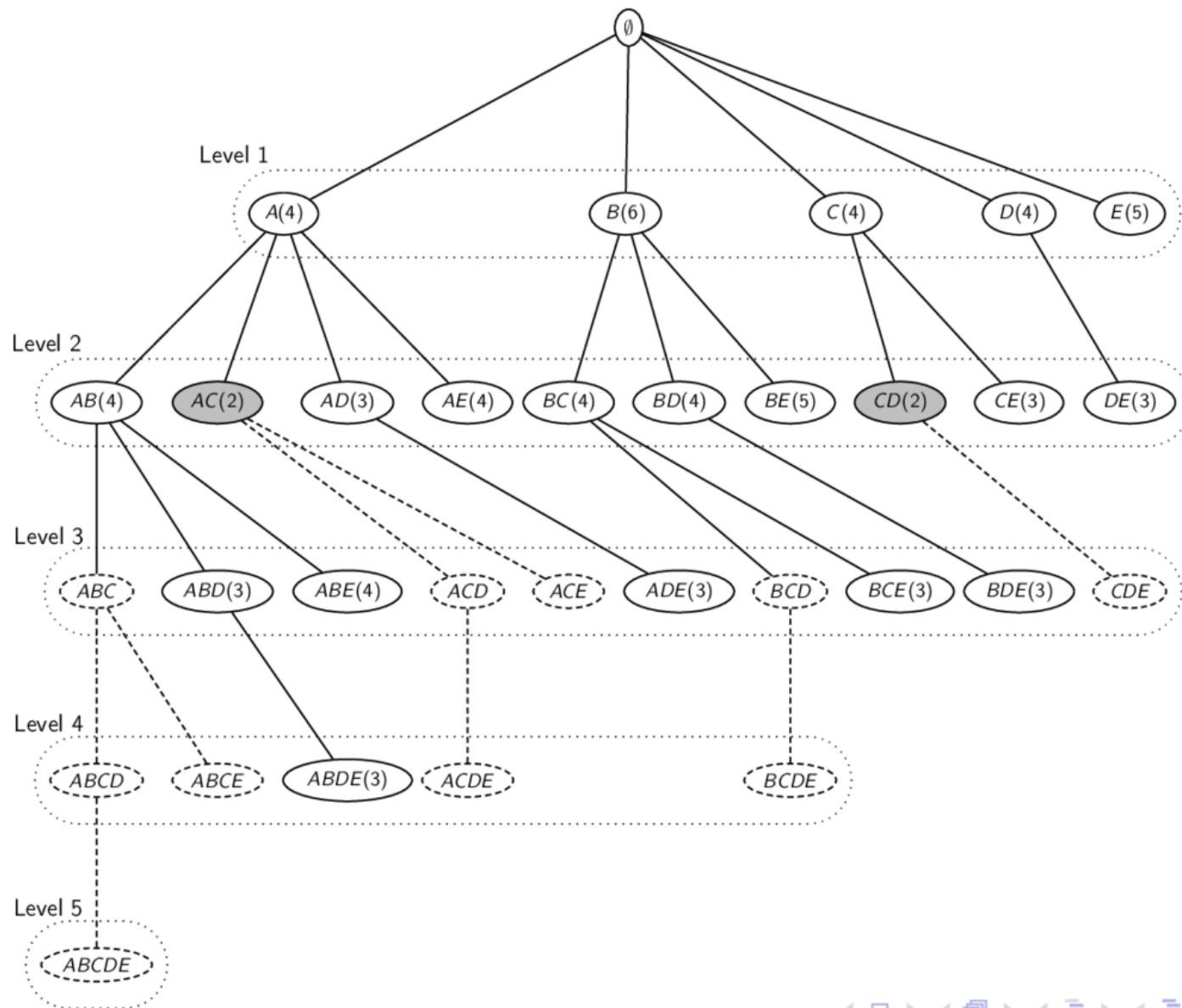
Infrequent itemsets in gray



Itemset Mining: Apriori Algorithm



Apriori Algorithm: Prefix Search Tree and Pruning



Apriori Algorithm: Details

Let $\mathcal{C}^{(k)}$ denote the prefix tree comprising all the candidate k -itemsets.

Apriori begins by inserting the single items into an initially empty prefix tree to populate $\mathcal{C}^{(1)}$.

The support for the current candidates is obtained via ComputeSupport procedure that generates k -subsets of each transaction in the database $\mathbf{D}$, and for each such subset it increments the support of the corresponding candidate in $\mathcal{C}^{(k)}$ if it exists. Next, we remove any infrequent candidate.

The leaves of the prefix tree that survive comprise the set of frequent k -itemsets $\mathcal{F}^{(k)}$, which are used to generate the candidate $(k + 1)$ -itemsets for the next level. The ExtendPrefixTree procedure employs prefix-based extension for candidate generation. Given two frequent k -itemsets X_a and X_b with a common $k - 1$ length prefix, that is, given two sibling leaf nodes with a common parent, we generate the $(k + 1)$ -length candidate $X_{ab} = X_a \cup X_b$. This candidate is retained only if it has no infrequent subset. Finally, if a k -itemset X_a has no extension, it is pruned from the prefix tree, and we recursively prune any of its ancestors with no k -itemset extension, so that in $\mathcal{C}^{(k)}$ all leaves are at level k .

If new candidates were added, the whole process is repeated for the next level. This process continues until no new candidates are added.

Association Rules

An *association rule* is an expression

$$X \xrightarrow{s,c} Y$$

where X and Y are itemsets and they are disjoint, that is, $X, Y \subseteq \mathcal{I}$, and $X \cap Y = \emptyset$. Let the itemset $X \cup Y$ be denoted as XY .

The *support* of the rule is the number of transactions in which both X and Y co-occur as subsets:

$$s = \text{sup}(X \longrightarrow Y) = |\mathbf{t}(XY)| = \text{sup}(XY)$$

The *relative support* of the rule is defined as the fraction of transactions where X and Y co-occur, and it provides an estimate of the joint probability of X and Y :

$$\text{rsup}(X \longrightarrow Y) = \frac{\text{sup}(XY)}{|\mathbf{D}|} = P(X \wedge Y)$$

The *confidence* of a rule is the conditional probability that a transaction contains Y given that it contains X :

$$c = \text{conf}(X \longrightarrow Y) = P(Y|X) = \frac{P(X \wedge Y)}{P(X)} = \frac{\text{sup}(XY)}{\text{sup}(X)}$$

Generating Association Rules

Given a frequent itemset $Z \in \mathcal{F}$, we look at all proper subsets $X \subset Z$ to compute rules of the form

$$X \xrightarrow{s,c} Y, \text{ where } Y = Z \setminus X$$

where $Z \setminus X = Z - X$.

The rule must be frequent because

$$s = \text{sup}(XY) = \text{sup}(Z) \geq \text{minsup}$$

We compute the confidence as follows:

$$c = \frac{\text{sup}(X \cup Y)}{\text{sup}(X)} = \frac{\text{sup}(Z)}{\text{sup}(X)}$$

If $c \geq \text{minconf}$, then the rule is a strong rule. On the other hand, if $\text{conf}(X \rightarrow Y) < c$, then $\text{conf}(W \rightarrow Z \setminus W) < c$ for all subsets $W \subset X$, as $\text{sup}(W) \geq \text{sup}(X)$. We can thus avoid checking subsets of X .